

Universidad Politécnica de Yucatán

High Performance Computing

Data Engineering 7B

**Activity: CA2.U2 Particle Simulator**

**Student:** Jose Angel Pech Xool

**Teacher:** Jose Francisco Perez Alcocer

February 9, 2026

# Introduction

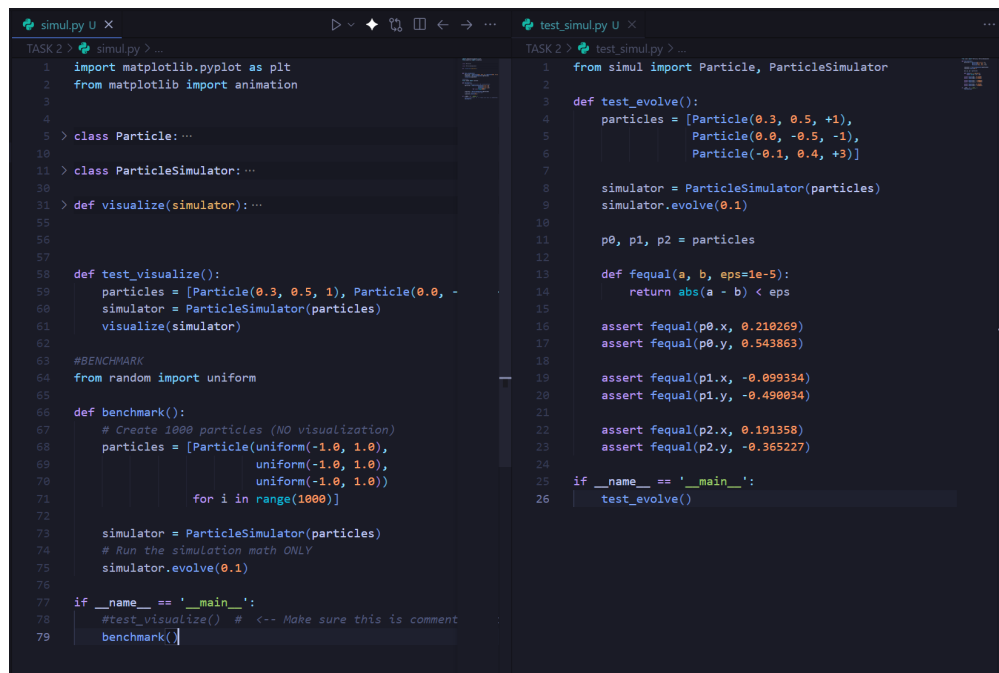
This technical report documents the benchmarking and profiling process of a particle simulation system, following the methodologies described in Chapter 1 of the "Benchmarking and Profiling" textbook (pp. 1-37). The objective is to identify performance bottlenecks using various Python tools (such as `cProfile`, `line_profiler`, and `memory_profiler`) and implement optimizations to improve execution time and memory usage.

## a) Writing Tests and Benchmarks

### Description:

Before optimization, it is critical to verify the correctness of the code. As stated in the textbook, "A solid test suite ensures that the implementation is correct at every iteration" (pp. 16). We implemented a `ParticleSimulator` class and a corresponding test suite using `pytest`. The test ensures that the simulation produces consistent results by comparing the final particle positions against known reference values (pp. 17). See Figure 1.

### Evidence:



```
simul.py U X
TASK 2 > simul.py > ...
1 import matplotlib.pyplot as plt
2 from matplotlib import animation
3
4
5 > class Particle: ...
10
11 > class ParticleSimulator: ...
30
31 > def visualize(simulator): ...
55
56
57
58 def test_visualize():
59     particles = [Particle(0.3, 0.5, 1), Particle(0.0, -
60     simulator = ParticleSimulator(particles)
61     visualize(simulator)
62
63 #BENCHMARK
64 from random import uniform
65
66 def benchmark():
67     # Create 1000 particles (NO visualization)
68     particles = [Particle(uniform(-1.0, 1.0),
69     uniform(-1.0, 1.0),
70     uniform(-1.0, 1.0))
71     for i in range(1000)]
72
73     simulator = ParticleSimulator(particles)
74     # Run the simulation math ONLY
75     simulator.evolve(0.1)
76
77 if __name__ == '__main__':
78     #test_visualize() # <-- Make sure this is comment
79     benchmark[]

test_simul.py U X
TASK 2 > test_simul.py > ...
1 from simul import Particle, ParticleSimulator
2
3 def test_evolve():
4     particles = [Particle(0.3, 0.5, +1),
5     Particle(0.0, -0.5, -1),
6     Particle(-0.1, 0.4, +3)]
7
8     simulator = ParticleSimulator(particles)
9     simulator.evolve(0.1)
10
11     p0, p1, p2 = particles
12
13     def fequal(a, b, eps=1e-5):
14         return abs(a - b) < eps
15
16     assert fequal(p0.x, 0.210269)
17     assert fequal(p0.y, 0.543863)
18
19     assert fequal(p1.x, -0.099334)
20     assert fequal(p1.y, -0.490034)
21
22     assert fequal(p2.x, 0.191358)
23     assert fequal(p2.y, -0.365227)
24
25 if __name__ == '__main__':
26     test_evolve()
```

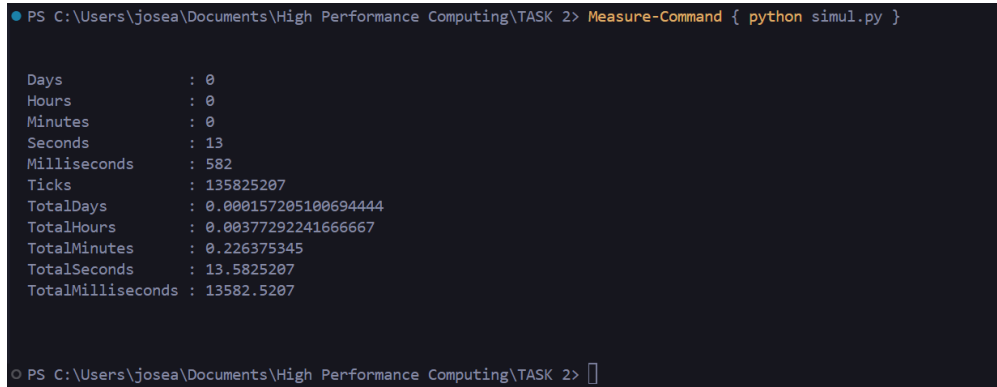
Figure 1: Source code for `simul.py` and `test_simul.py`

## b) Timing Benchmark with Unix Command

### Description:

We established a baseline execution time using the system's time measurement tool. The textbook notes that for Windows environments, "you can use similar PowerShell commands, such as Measure-Command" (pp. 18). This command measures the real execution time, serving as a simple and direct way to benchmark the program (pp. 19). The results can be seen in Figure 2.

#### Evidence:



```
PS C:\Users\josea\Documents\High Performance Computing\TASK > Measure-Command { python simul.py }

Days           : 0
Hours          : 0
Minutes        : 0
Seconds        : 13
Milliseconds   : 582
Ticks          : 135825207
TotalDays      : 0.000157205100694444
TotalHours     : 0.00377292241666667
TotalMinutes   : 0.226375345
TotalSeconds   : 13.5825207
TotalMilliseconds : 13582.5207

PS C:\Users\josea\Documents\High Performance Computing\TASK > 
```

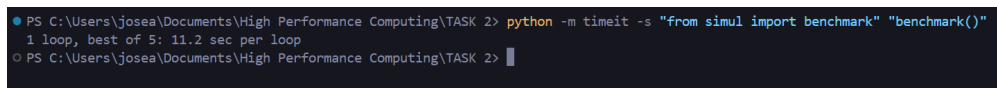
Figure 2: Baseline timing showing approx. 13.58 seconds execution

### c) Benchmark using timeit

#### Description:

For a more rigorous measurement, we used the `timeit` module. Unlike the system time command, `timeit` runs the code in a loop multiple times to minimize system overhead and provide a statistically valid best-case execution time (pp. 19). This method avoids outliers that might skew the results (pp. 20). See Figure 3.

#### Evidence:



```
PS C:\Users\josea\Documents\High Performance Computing\TASK > python -m timeit -s "from simul import benchmark" "benchmark()"
1 loop, best of 5: 11.2 sec per loop
PS C:\Users\josea\Documents\High Performance Computing\TASK > 
```

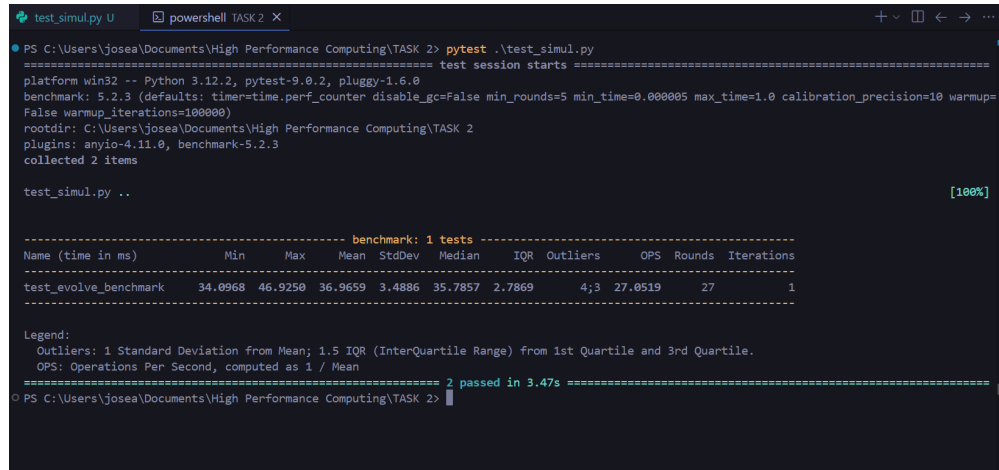
Figure 3: Benchmarking results using the timeit module

### d) Benchmarking with pytest-benchmark

#### Description:

We integrated the benchmarking process directly into the test suite using the `pytest-benchmark` plugin. As described in the text, this allows us to "validate the correctness of the physics engine (via assertions) and measure performance statistics (Min, Max, Mean) in a single execution step" (pp. 21). This tool provides a statistical summary that highlights performance variability between runs (pp. 23). Figure 4 shows the successful test results.

## Evidence:



```
test_simul.py U powershell TASK 2 X
PS C:\Users\josea\Documents\High Performance Computing\TASK 2> pytest .\test_simul.py
===== test session starts =====
platform win32 -- Python 3.12.2, pytest-9.0.2, pluggy-1.6.0
benchmark: 5.2.3 (defaults: timer=time.perf_counter disable_gc=False min_rounds=5 min_time=0.000005 max_time=1.0 calibration_precision=10 warmup=False warmup_iterations=100000)
rootdir: C:\Users\josea\Documents\High Performance Computing\TASK 2
plugins: anyio-4.11.0, benchmark-5.2.3
collected 2 items

test_simul.py .. [100%]

----- benchmark: 1 tests -----
Name (time in ms)      Min      Max     Mean  StdDev   Median    IQR  Outliers    OPS    Rounds  Iterations
-----
test_evolve_benchmark 34.0968  46.9250  36.9659  3.4886   35.7857   2.7869      4;3    27.0519      27         1

Legend:
  Outliers: 1 Standard Deviation from Mean; 1.5 IQR (InterQuartile Range) from 1st Quartile and 3rd Quartile.
  OPS: Operations Per Second, computed as 1 / Mean
===== 2 passed in 3.47s =====
PS C:\Users\josea\Documents\High Performance Computing\TASK 2>
```

Figure 4: Pytest results showing passed tests and benchmark statistics

## e) Finding Bottlenecks with cProfile

### Description:

To locate the performance bottleneck, we used the deterministic profiler `cProfile`. This module "is written in C, has a small overhead, and is suitable as a general purpose profiler" (pp. 24). By sorting the output by "total time" (`tottime`), we identified that the `evolve` function was responsible for the majority of the execution time (pp. 24). See Figure 5.

### Evidence:

```
PS C:\Users\josea\Documents\High Performance Computing\TASK 2> python -m cProfile -s tottime simul.py | select -first 20
1094886 function calls (1079770 primitive calls) in 13.587 seconds

Ordered by: internal time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      11.162    11.162    11.162    11.162 simul.py:15(evolve)
295     0.256     0.001     0.256     0.001 {built-in method _io.open_code}
7176    0.179     0.000     0.341     0.000 inspect.py:879(cleandoc)
1299    0.154     0.000     0.155     0.000 {built-in method nt.stat}
295     0.112     0.000     0.112     0.000 {built-in method marshal.loads}
218     0.066     0.000     0.204     0.001 artist.py:1441(get_aliases)
171207/170102 0.055     0.000     0.056     0.000 {built-in method builtins.len}
99217   0.055     0.000     0.055     0.000 {method 'startswith' of 'str' objects}
20      0.054     0.003     0.072     0.004 {built-in method _imp.create_dynamic}
9617    0.051     0.000     0.051     0.000 {method 'search' of 're.Pattern' objects}
295     0.050     0.000     0.050     0.000 {method 'read' of '_io.BufferedReader' objects}
218     0.044     0.000     0.133     0.001 artist.py:1519(get_setters)
86119   0.044     0.000     0.044     0.000 {method 'rstrip' of 'str' objects}
438     0.043     0.000     0.043     0.000 {built-in method builtins.dir}
295     0.040     0.000     0.154     0.001 <frozen importlib._bootstrap_external>:751(_compile_bytecode)
PS C:\Users\josea\Documents\High Performance Computing\TASK 2>
```

Figure 5: cProfile output sorted by total time

## f) Visualization with SnakeViz

### Description:

To better understand the call stack, we visualized the profiling data. The textbook suggests tools like `KCachegrind` to analyze profiling output graphically (pp. 26-27). In this report, we utilized `SnakeViz` as a modern alternative for visualizing the call hierarchy, which helps interpret complex call stacks that text output might obscure (pp. 28). Figure 6 shows the visualization.

### Evidence:

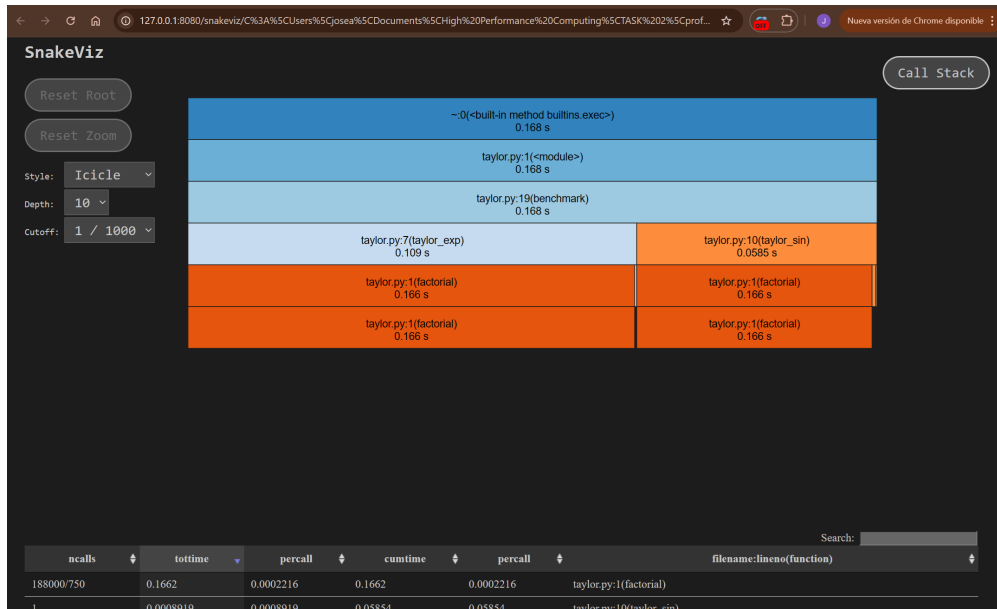


Figure 6: SnakeViz visualization of the profiling data

## g) Line-by-Line Profiling

### Description:

Since `cProfile` only identifies the function level, we used `line_profiler` (`kernprof`) to analyze the `evolve` function line-by-line. As noted in the text, "this is very useful in situations where it's difficult to determine which statements are costly" (pp. 29). The results in Figure 7 confirmed that the distance calculation (involving square roots) inside the loop was the most expensive operation (pp. 31).

### Evidence:

```

PS C:\Users\josea\Documents\High Performance Computing\TASK 2> kernprof -l -v simul_line.py
Wrote profile results to 'simul_line.py.lprof'
Timer unit: 1e-06 s

Total time: 124.653 s
File: simul_line.py
Function: ParticleSimulator.evolve at line 15

Line #      Hits          Time Per Hit   % Time  Line Contents
=====
15                                     @profile
16                                     def evolve(self, dt):
17             1             3.2       3.2      0.0         timestep = 0.00001
18             1             5.1       5.1      0.0         nsteps = int(dt/timestep)
19          10001          18321.8       1.8      0.0         for i in range(nsteps):
20         10010000         14088418.0       1.4     11.3             for p in self.particles:
21                                     #calculate the distance from the center
22          10000000         22744835.5       2.3     18.2                 r = (p.x**2 + p.y**2)**0.5
23          10000000         14630136.1       1.5     11.7                 sin_theta = p.y / r
24          10000000         14103970.8       1.4     11.3                 cos_theta = p.x / r
25                                     #calculate the displacemente
26          10000000         14907551.4       1.5     12.0                 dx = -timestep*p.v*sin_theta
27          10000000         14508831.9       1.5     11.6                 dy = timestep*p.v*cos_theta
28                                     #update the position
29          10000000         14909618.5       1.5     12.0                 p.x += dx
30          10000000         14741318.5       1.5     11.8                 p.y += dy

```

PS C:\Users\josea\Documents\High Performance Computing\TASK 2> █

Figure 7: Line profiler results showing % Time per line

## h) Code Optimization

### Description:

Based on the profiling results, we optimized the code by moving invariant calculations outside the loop and simplifying the coordinate update logic to avoid square root calculations, as suggested in the "Optimizing our code" section (pp. 31-32).

### Optimized Code Snippet:

```
1 # Optimization: Pre-calculate angular factor and remove square roots
2 for p in self.particles:
3     t_x_ang = timestep * p.ang_vel
4     for i in range(nsteps):
5         norm = (p.x**2 + p.y**2)**0.5
6         p.x, p.y = (p.x - t_x_ang * p.y/norm,
7                     p.y + t_x_ang * p.x/norm)
```

### Evidence:

The optimized version shows a clear performance improvement (Figure 8), consistent with the "modest increment in speed" described in the text (pp. 32).

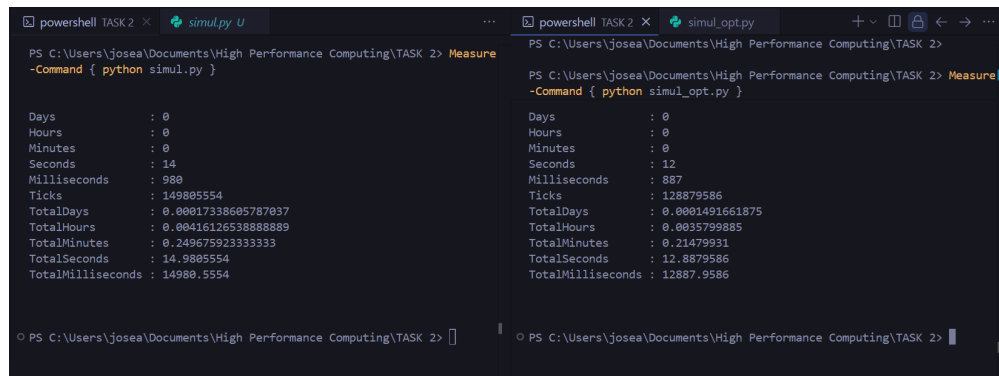


Figure 8: Comparison: Original vs. Optimized execution times

## j) Memory Profiling

### Description:

We analyzed the memory consumption of the `Particle` objects using the `memory_profiler` tool (pp. 34). To reduce the memory footprint, we utilized the `__slots__` attribute. As explained in the textbook, "This feature saves some memory by avoiding storing the variables of the instance in an internal dictionary" (pp. 36).

### Code Implementation:

```
1 class Particle:
2     __slots__ = ('x', 'y', 'v') # Reduces memory overhead
```



```

3  def __init__(self, x, y, v):
4      self.x = x
5      self.y = y
6      self.v = v

```

## Evidence:

Figure 9 shows the baseline memory usage. Figure 10 shows the reduced memory usage after the optimization. Note: The memory reduction is smaller than in the textbook because modern Python versions (3.12) include built-in dictionary optimizations that lower the baseline memory footprint.

```

PS C:\Users\josea\Documents\High Performance Computing\TASK 2> python -m memory_profiler simul_mem.py
Filename: simul_mem.py

Line #    Mem usage    Increment   Occurrences   Line Contents
=====
26  54.668 MiB   54.668 MiB         1   @profile
27
28      def benchmark_memory():
29          # We use 100,000 particles to make the memory usage visible
30          particles = [Particle(uniform(-1.0, 1.0),
31                               uniform(-1.0, 1.0),
32                               uniform(-1.0, 1.0))
33                        for i in range(100000)]
34
35      simulator = ParticleSimulator(particles)
36      simulator.evolve(0.001)

PS C:\Users\josea\Documents\High Performance Computing\TASK 2>

```

Figure 9: Baseline memory usage (Total Increment approx 19.9 MiB)

```

PS C:\Users\josea\Documents\High Performance Computing\TASK 2> python -m memory_profiler simul_mem.py
Filename: simul_mem.py

Line #    Mem usage    Increment   Occurrences   Line Contents
=====
27  54.648 MiB   54.648 MiB         1   @profile
28      def benchmark_memory():
29          # We use 100,000 particles to make the memory usage visible
30          particles = [Particle(uniform(-1.0, 1.0),
31                               uniform(-1.0, 1.0),
32                               uniform(-1.0, 1.0))
33                        for i in range(100000)]
34
35      simulator = ParticleSimulator(particles)
36      simulator.evolve(0.001)

PS C:\Users\josea\Documents\High Performance Computing\TASK 2>

```

Figure 10: Optimized memory usage with `__slots__` (Total Increment approx 16.5 MiB)